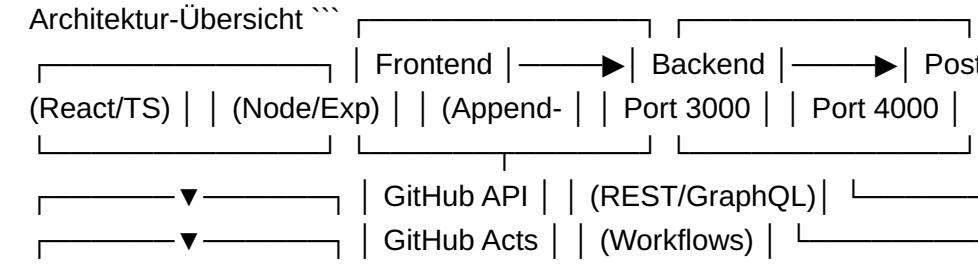
Anhang A9 — Entwicklerdokumentation **Projekt:** Zero-Trust-Sicherheitskonzept mit GitHub-Integration **Repository:** `vfb-bildung/zero-trust-rbac` (private) **Version:** 1.0 | **Datum:** 09.07.2026 --- ## 1. Quickstart ### Voraussetzungen - Docker & Docker Compose v2+ - Node.js 18+ (für lokale Frontend-Entwicklung) - PostgreSQL 15+ (oder via Docker) - GitHub Personal Access Token (PAT) mit `admin:org`, `read:org`, `workflow` Scopes ### Starten der Entwicklungsumgebung ```bash # Repository klonen git clone https://github.com/vfb-bildung/zero-trust-rbac.git cd zero-trust-rbac # Environment konfigurieren cp .env.example .env # .env bearbeiten: GITHUB_TOKEN, DB_PASSWORD, JWT_SECRET, etc. # Services starten docker-compose up -d # Frontend separat (Hot Reload) cd frontend && npm install && npm run dev ``` ### Services | Service | Port | Beschreibung | |---------|------|--------------| | Frontend (React) | 3000 | Self-Service-Portal | | Backend (Node/Express) | 4000 | REST API, Webhooks | | PostgreSQL | 5432 | Persistenz | | Redis | 6379 | Cache (Berechtigungen) | | GitHub Actions | — | CI/CD, Provisioning | --- ## 2. Architektur-Übersicht ```


``` --- ## 3. Backend API (OpenAPI 3.0) ### Base URL ``` http://localhost:4000/api/v1 ``` ### Authentifizierung - **SSO:** Azure AD / SAML → Session Cookie - **API:** JWT Bearer Token (`Authorization: Bearer <token>`) ### Endpunkte | Methode | Pfad | Beschreibung | Auth | |---------|------|--------------|------| | `GET` | `/health` | Health Check | — | | `POST` | `/auth/login` | SSO-Callback | — | | `GET` | `/me` | Aktueller Nutzer + Rollen | JWT | | `GET` | `/roles` | Rollen-Katalog | JWT | | `POST` | `/requests` | Rollenantrag stellen | JWT | | `GET` | `/requests` | Eigene Anträge (paginiert) | JWT | | `GET` | `/requests/:id` | Antragsdetail | JWT | | `POST` | `/approvals` | Genehmigung/Abteilung | JWT (VG) | | `GET` | `/audit-logs` | Audit-Logs (Filter) | JWT (Admin) | | `POST` | `/export` | CSV/JSON Export | JWT (Admin) | ### Beispiel: Rollenantrag stellen ```bash curl -X POST http://localhost:4000/api/v1/requests \ -H "Authorization: Bearer <token>" \ -H "Content-Type: application/json" \ -d '{ "roleId": "uuid-of-developer-role", "reason": "Need write access for frontend repo", "resource": "vfb-bildung/frontend" }' ``` **Response 201:** ```json { "id": "uuid", "status": "pending", "requestedAt": "2026-07-09T10:15:00Z", "role": { "id": "...", "name": "Developer" } } ``` --- ## 4. GitHub Actions Workflow ### Datei: `.github/workflows/role-request.yml` ```yaml name: Role Request Workflow on: issues: types: [opened, labeled] jobs: validate: runs-on: ubuntu-latest steps: - name: Check required fields run: | # Prüfung: role, reason, resource vorhanden? - name: Policy check run: | # 4-Augen-Prinzip, Kompetenzmatrix prüfen # Blockiert bei Verstoß approve: needs: validate runs-on: ubuntu-latest environment: approval steps: - name: Request approval uses: actions/github-script@v7 with: script: | // Erstellt Genehmigungs-Review an Vorgesetzten // 48h Timeout → Eskalation provision: needs: approve if: ${{ needs.approve.outputs.approved == 'true' }} runs-on: ubuntu-latest steps: - name: Assign GitHub Team run: | gh api --method PUT \ /orgs/vfb-bildung/teams/$
```

{{ needs.approve.outputs.team_slug }}/memberships/${{ github.actor }} - name: Create Audit Log run: | # INSERT INTO audit_log ... - name: Notify requester run: | # E-Mail + GitHub Notification notify: needs: [provision, approve] if: always() runs-on: ubuntu-latest steps: - name: Send status notification run: | # E-Mail + GitHub Notification (approved/rejected/failed) ``` ### Secrets (GitHub Repository Settings → Secrets) | Secret | Beschreibung | |--------|--------------| | `GITHUB_TOKEN` | Auto-provided (workflow scope) | | `GH_PAT_ADMIN` | PAT mit `admin:org` für Team-Management | | `DB_URL` | PostgreSQL Connection String | | `JWT_SECRET` | Signing Key für API-Tokens | | `AZURE_AD_CLIENT_ID` | SSO Client ID | | `AZURE_AD_TENANT_ID` | SSO Tenant ID | --- ## 5. Frontend (React + TypeScript) ### Struktur ``` frontend/ ├── src/ │ ├── components/ # Wiederverwendbare UI-Komponenten │ ├── pages/ # Seiten (Dashboard, Antrag, Admin, etc.) │ ├── hooks/ # Custom Hooks (useAuth, useRequests, etc.) │ ├── services/ # API-Client (Axios + Interceptors) │ ├── types/ # TypeScript Interfaces │ ├── utils/ # Helpers (Formatierung, Validierung) │ └── App.tsx # Router + Provider ├── package.json ├── tsconfig.json ├── vite.config.ts └── .env.example ``` ### Wichtige Komponenten | Komponente | Beschreibung | |------------|--------------| | `RoleRequestForm` | Antragsformular mit Validierung | | `RequestStatusCard` | Statusanzeige mit Ampel | | `RoleBadge` | Rollenanzeige mit Berechtigungen | | `AuditLogTable` | Sortierbare, filterbare Tabelle | | `ApprovalModal` | Genehmigen/Ablehnen Dialog | | `AdminRoleMatrix` | Editierbare Matrix (Checkbox-Grid) | ### State Management - **Auth:** Context + `useAuth()` Hook (JWT in HttpOnly Cookie) - **Server State:** TanStack Query (React Query) für Caching - **UI State:** React `useState` / `useReducer` --- ## 6. Datenbank-Migrationen ```bash # Migration erstellen npx knex migrate:make create_audit_log_table # Migrationen ausführen npx knex migrate:latest # Rollback npx knex migrate:rollback ``` ### Beispiel-Migration: Audit-Log ```javascript // migrations/20260709_create_audit_log.js exports.up = async (knex) => { await knex.schema.createTable('audit_log', (t) => { t.bigIncrements('id').primary(); t.timestamp('timestamp').defaultTo(knex.fn.now()); t.uuid('user_id').references('id').inTable('user').onDelete('SET NULL'); t.string('action', 50).notNullable(); t.string('resource_type', 50).notNullable(); t.uuid('resource_id').nullable(); t.string('result', 20).notNullable().checkIn(['success','failed','pending']); t.jsonb('details'); t.char('previous_hash', 64); t.char('current_hash', 64).notNullable(); }); await knex.raw(` CREATE INDEX idx_audit_log_timestamp ON audit_log(timestamp DESC); CREATE INDEX idx_audit_log_user ON audit_log(user_id); `); }; exports.down = (knex) => knex.schema.dropTable('audit_log'); ``` --- ## 6. Tests ```bash # Unit Tests (Jest) npm test ---coverage # Integration Tests (Supertest) npm run test:integration # E2E Tests (Playwright) npm run test:e2e # Security Scan (GitHub CodeQL) # Läuft automatisch in CI ``` ### Testfall-Beispiel (Integration) ```typescript // tests/integration/role-request.test.ts describe('Role Request Flow', () => { it('should create request, approve, and provision', async () => { // 1. User creates request const request = await request(app) .post('/api/v1/requests') .set('Authorization', `Bearer ${userToken}`) .send({ roleId: developerRoleId, reason: 'Test' }) .expect(201); // 2. Approver approves await request(app) .post('/api/v1/approvals') .set('Authorization', `Bearer ${approverToken}`) .send({ requestId:

request.body.id, decision: 'approve' }) .expect(200); // 3. Verify GitHub team membership const membership = await github.teams.getMembershipForUser(...); expect(membership.state).toBe('active'); // 4. Verify audit log const logs = await db('audit_log').where('action', 'GRANT'); expect(logs).toHaveLength(1); }); }); ``` --- ## 7. Deployment (UAT / Produktion) ### Docker Compose (Produktion) ```yaml # docker-compose.prod.yml services: frontend: image: vfb/zero-trust-frontend:latest ports: ["80:80"] depends_on: [backend] backend: image: vfb/zero-trust-backend:latest environment: - NODE_ENV=production - DB_URL=${DB_URL} - JWT_SECRET=${JWT_SECRET} deploy: replicas: 2 db: image: postgres:15 volumes: [pgdata:/var/lib/postgresql/data] environment: - POSTGRES_PASSWORD=${DB_PASSWORD} ``` ### CI/CD Pipeline 1. **Push to main** → CodeQL Scan + Unit Tests 2. **Merge to main** → Build Docker Images → Push to GHCR 3. **Tag Release** → Deploy to UAT (staging) 3. **Manual Approval** → Deploy to Production --- ## 8. Troubleshooting | Problem | Ursache | Lösung | |---------|---------|--------| | `401 Unauthorized` | JWT abgelaufen | Refresh Token / Re-Login | | `GitHub API 403` | PAT fehlende Scopes | `admin:org`, `workflow` prüfen | | `DB Connection refused` | PostgreSQL nicht bereit | `docker-compose up -d db` warten | | `GitHub Actions: 422` | Workflow-YAML Syntax | `yamllint .github/workflows/*.yml` | | `Rate Limit` | GitHub API Limit | `gh api rate_limit` prüfen, Caching | --- ## 9. Nützliche Befehle ```bash # Logs anzeigen docker-compose logs -f backend # Datenbank-Shell docker-compose exec db psql -U postgres -d zero_trust # Migrationen manuell docker-compose exec backend npx knex migrate:latest # Tests in Container docker-compose exec backend npm test # Frontend Build cd frontend && npm run build # Audit-Log Export (Admin) curl -H "Authorization: Bearer $ADMIN_TOKEN" \ "http://localhost:4000/api/v1/export?format=csv&from=2026-07-01" > audit.csv ``` --- *Ende Anhang A9. Vollständiger Code im Repository `vfb-bildung/zero-trust-rbac` (private).*